📄 Symplichain Hackathon Submission

Role: Software Engineering Intern
Candidate: Vamsi Deverakonda

---

1. Introduction

This submission proposes practical and scalable solutions for rate-limiting shared APIs, mobile architecture design, CI/CD improvements, and systematic debugging. The focus is on leveraging the existing stack (Django, React, AWS, Redis, Celery) while keeping the system cost-efficient and maintainable.

---

2. Part 1: Shared Gateway Problem (Rate Limiting System)

🔹 Problem Summary

- External API limit: **3 requests/second**
- Current load: **20+ requests/second**
- Goal: Enforce limit + ensure fairness across customers

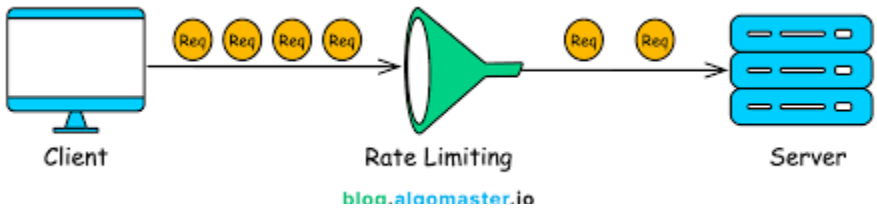
🔹 Proposed Architecture

Client Requests
↓
API Gateway (Django)
↓
Redis Queue (per customer)
↓
Rate Limiter (Token Bucket)
↓

Celery Workers
↓
External API

🔹 Components Used

- **Redis** → Queue + rate limiting
- **Celery** → Async processing
- **Django** → Request intake layer
- **AWS EC2** → Hosting workers



🔹 Rate Enforcement Strategy

✅ Token Bucket Algorithm

- Bucket capacity = 3 tokens
- Refill rate = 3 tokens/sec
- Each request consumes 1 token

Guarantees: **Max 3 requests/sec**

🔹 Fairness Strategy

❌ Problem:

Customer A sends 100 requests
Customer B sends 1 request

Without control → B waits too long ❌

✅ Solution: Round-Robin Scheduling

- Maintain separate queues per customer
- Process requests in rotation:

A → B → C → A → B → C

🔹 Ensures:

- No starvation
- Equal opportunity

🔹 Failure Handling

Retry Strategy:

- Retry only on 5xx errors
- Use Exponential Backoff:
  - 1s → 2s → 4s
- Max retries: 3

Dead Letter Queue:

- Failed tasks stored for inspection

🔹 Why This Approach?

- Uses existing Redis + Celery
- No major infra cost
- Scales horizontally

---

3. Part 2: Mobile App Architecture

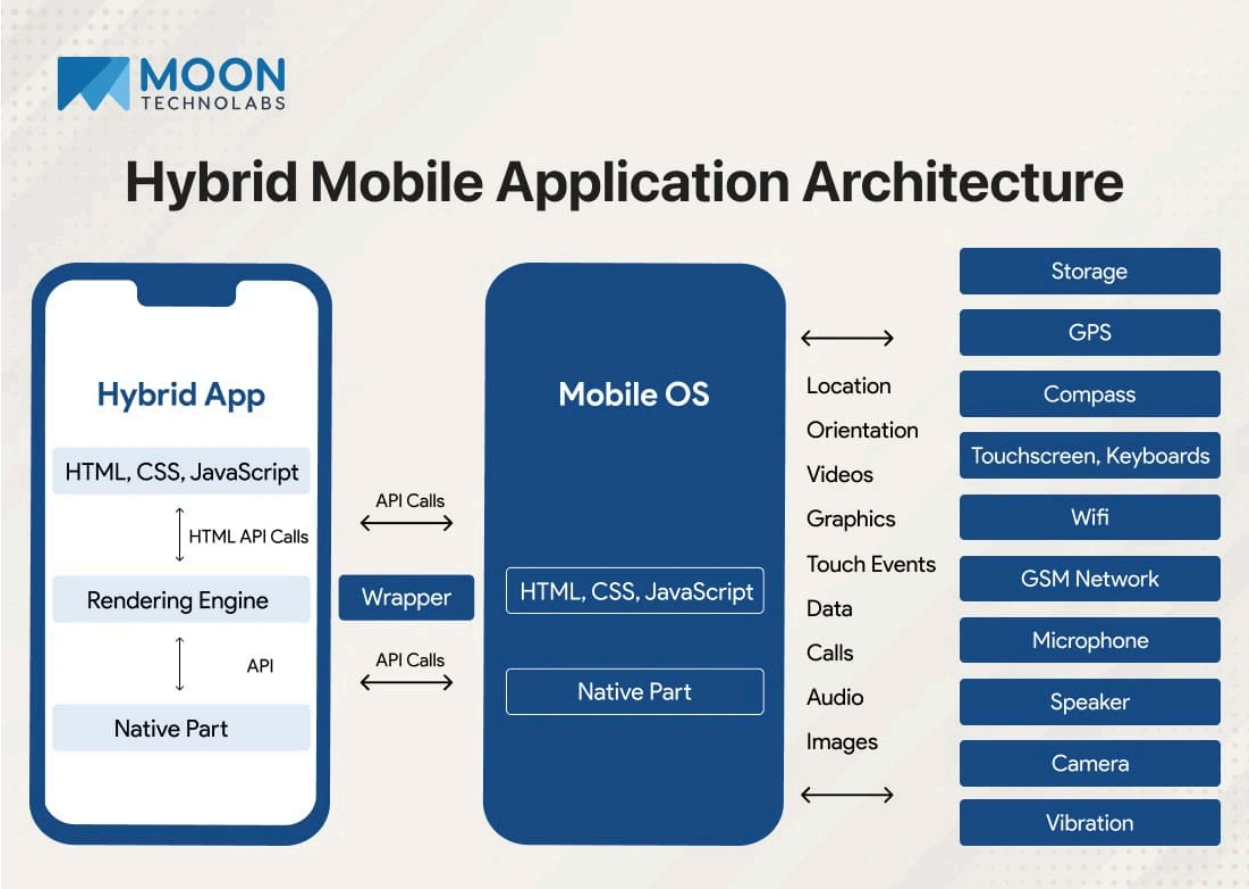
🔹 Interaction Model

✅ Primary:

- **Click-based UI (fast, reliable)**

🔹 **Optional:**

- **AI Assistant (future enhancement)**



🔹 Tech Stack Choice

✅ React Native

Reasons:

- **Team already uses React**
- **Faster development**
- **Single codebase for Android + iOS**
- **Lower cost vs native**

🔹 Architecture

Mobile App (React Native)

↓

API Gateway (Django REST API)

↓

Backend Services

↓

Database (PostgreSQL)

🔹 Optimizations

- API caching
- Lazy loading
- Minimal steps UX
- Offline support (basic caching)

---

4. Part 3: CI/CD Pipeline

🔹 Current Flow

- Push → GitHub → Deploy EC2

🔹 Proposed GitHub Actions Workflow

```
name: Deploy Application


on:

  push:
```

```
    branches:
      - staging
      - main

jobs:
  deploy:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout code
        uses: actions/checkout@v3

      - name: Setup Python
        uses: actions/setup-python@v4
        with:
          python-version: 3.10

      - name: Install Backend Dependencies
        run: |
          pip install -r requirements.txt

      - name: Build Frontend
        run: |
          cd frontend
```

```
    npm install
    npm run build

  - name: Deploy to EC2
    uses: appleboy/ssh-action@v0.1.6
    with:
      host: ${{ secrets.EC2_HOST }}
      username: ubuntu
      key: ${{ secrets.EC2_SSH_KEY }}
      script: |
        cd /var/www/app
        git pull origin ${{ github.ref_name }}
        pip install -r requirements.txt
        cd frontend
        npm install
        npm run build
        sudo systemctl restart gunicorn
        sudo systemctl restart nginx
```

🔹 Improvements

✅ Docker

- Consistent environments
- Easier deployment

✅ Terraform

- Automate AWS setup

- Remove manual steps

---

5. Part 4: Debugging – Monday Outage

🔹 Problem

POD image upload failing

Flow:

Driver App → Django → S3 → Celery → Bedrock → RDS

🔹 Step-by-Step Debugging

✅ Step 1: Check Django Logs

Why? Entry point of failure

Command:

sudo journalctl -u gunicorn

Check:

- Request errors
- API failures

✅ Step 2: Check S3 Upload

Why? Verify file storage

- Open AWS S3 console
- Check if images exist

✅ Step 3: Check Celery Tasks

Why? Async processing

- **Use Flower dashboard**
- **Check:**
  - **Task status**
  - **Failures**

✅ Step 4: Check Bedrock API

Why? AI validation step

- **Look at logs in CloudWatch**
- **Check API response errors**

✅ Step 5: Check PostgreSQL (RDS)

Why? Final output storage

Run query: SELECT * FROM pod_table ORDER BY created_at DESC;

🔹 Possible Root Causes

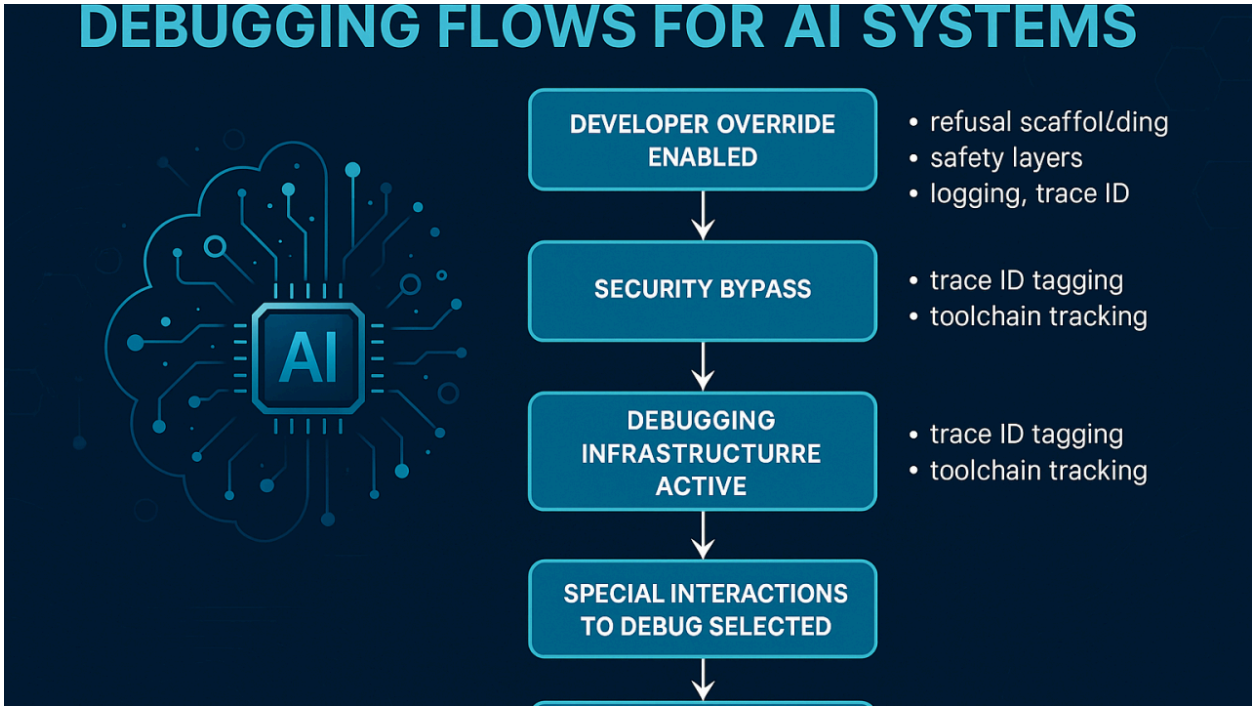
- **S3 permission issue**
- **Celery worker down**
- **Bedrock API failure**
- **DB connection issue**

---

6. Assumptions & Improvements

🔹 Assumptions

- **Redis and Celery already configured**
- **AWS services accessible**
- **API failures are intermittent**



🔹 Future Improvements

- **Add centralized monitoring (Prometheus + Grafana)**
- **Add auto-scaling workers**
- **Implement model serving pipeline**
- **Full Infrastructure as Code (Terraform)**

✅ Final Notes

This solution focuses on:

- **Simplicity**
- **Cost-efficiency**
- **Reuse of existing infrastructure**

🔹 **“Why I chose this design”?**

I prioritized using Redis and Celery since they are already part of the existing stack, avoiding additional infrastructure cost. I chose simple and scalable solutions like token bucket and round-robin instead of complex distributed systems to ensure maintainability and quick implementation.